LightPrune: Latency-Aware Structured Pruning for Efficient Deep Inference on Embedded Devices

Asma Belhadi
OsloMet University
Oslo, Norway
asma.belhadi@oslomet.no

Youcef Djenouri
NORCE Norwegian Research Center
Oslo, Norway
yodj@norceresearch.no

Ahmed Nabil Belbachir
NORCE Norwegian Research Center
Grimstad, Norway
nabe@norceresearch.no

2025 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW) | 979-8-3315-8988-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCVW69036.2025.00179

Abstract

In this work, we present LightPrune, a structured pruning framework tailored for deploying deep convolutional neural networks (CNNs) on resource-constrained embedded devices. As the demand for intelligent vision applications grows in mobile and IoT platforms, the challenge of balancing accuracy, model compactness, and real-time latency becomes critical. Existing pruning methods often emphasize model compression or sparsity but fail to explicitly optimize for inference latency, particularly on target hardware. LightPrune introduces a latency-aware progressive pruning strategy that jointly accounts for structured sparsity, hardware feedback, and training stability. We integrate a differentiable latency estimator into the pruning process to guide filter removal based on real execution cost, not proxy FLOPs or MACs. Furthermore, we adopt a progressive pruning schedule with regularization-based sparsity control to iteratively refine the network while preserving accuracy. Extensive experiments on CIFAR-100, Tiny-ImageNet, and ImageNet-100 using MobileNetV2 show that LightPrune achieves up to 3× reduction in model size and 2.2× speed-up in latency on Jetson Nano, with less than 1% accuracy drop. Our method outperforms existing structured pruning baselines, offering a practical and scalable solution for edge AI deployment.

1. Introduction

The remarkable advances in convolutional neural networks (CNNs) have redefined the landscape of computer vision, enabling breakthroughs in tasks such as object recognition [22], scene understanding [7], and real-time tracking [2]. Despite these achievements, deploying state-of-the-art deep models on edge and embedded platforms—such as drones, mobile phones, and microcontroller-powered sensors—remains a significant challenge [1, 6, 15]. These devices are typically constrained by limited computational power, memory bandwidth, and energy budgets. As a result, enabling high-performance inference under strict resource constraints has become a fundamental bottleneck in the broader adoption of vision intelligence at the edge.

Model pruning has emerged as a promising solution to this problem by eliminating redundant parameters in over-parameterized networks [4, 20, 37]. However, most existing pruning methods focus primarily on reducing the number of parameters or FLOPs, assuming these to be strong proxies for latency or energy consumption. Yet, in practice, this assumption often breaks down: reductions in theoretical complexity do not necessarily translate to real latency gains on modern embedded systems, where execution time is influenced by memory access patterns, hardware accelerators, and kernel-level optimizations. Moreover, many pruning frameworks adopt a one-shot approach, compressing the model in a single step and often yielding sub-optimal trade-offs between accuracy and efficiency [16, 35, 41]. These methods typically disregard the dynamics of network retraining, the impact of pruning order, and the potential instability introduced by aggressive filter removal. Furthermore, they rarely account for deployment-time latency measurements, leading to pruned models that may be compact but still incompatible with real-time constraints.

In this paper, we introduce LightPrune, a principled pruning framework tailored for real-time deployment of CNNs on embedded hardware. LightPrune is built around three foundational insights. First, latency should be treated

2473-9944/25/$31.00 ©2025 IEEE
DOI 10.1109/ICCVW69036.2025.00179

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.

as a first-class optimization objective, rather than approximated through FLOPs or model size. Second, structured pruning is preferable to unstructured sparsity in edge scenarios, as it produces models more compatible with hardware acceleration. Third, pruning should be performed progressively, allowing the model to iteratively adapt and recover capacity after each pruning stage, ultimately preserving predictive accuracy. LightPrune integrates these insights into a unified optimization framework. It combines structured filter pruning with a progressive schedule, guided by hardware-in-the-loop latency estimation. At each stage, filters are ranked based not only on saliency but also on their contribution to real latency. A sparsity-inducing regularization term further encourages the formation of compact architectures without harsh constraints. Our method is general, lightweight, and applicable to a wide range of CNN architectures and edge devices.

Our main contributions can be summarized as follows:

1. We propose LightPrune, a latency-aware, structured pruning framework explicitly designed for edge deployment.
2. We develop a progressive pruning schedule coupled with hardware-in-the-loop latency estimation, optimizing not just compression but real execution efficiency.
3. We design a joint loss incorporating sparsity regularization and latency constraints, balancing compactness and performance.
4. We conduct a thorough evaluation across three datasets and a well-known hardware-efficient architecture, demonstrating LightPrune's superiority in maintaining accuracy under tight resource budgets.

2. Problem Formulation

The proliferation of deep neural networks (DNNs) has led to remarkable advancements in computer vision tasks. Nevertheless, their deployment on embedded platforms such as drones, mobile devices, and edge processors remains severely limited by constraints on memory, energy, and real-time responsiveness. To address these limitations, pruning has emerged as a critical technique for model compression, wherein the goal is to identify and remove redundant or less important components—such as filters, channels, or neurons—while preserving task performance.

Consider a pretrained neural network $\mathcal{M}$ comprising n layers $L_1, L_2, \ldots, L_n$. Each layer L_i has c_i output channels. We associate with each layer a learnable importance vector $\boldsymbol{s}^{(i)} \in \mathbb{R}^{c_i}$, where $s_j^{(i)} \in [0, 1]$ reflects the relative importance of the j-th channel in layer i. These importance scores act as continuous masks that scale the output of corresponding channels. During training, these masks are used to gradually identify which channels are dispensable. The pruning objective must simultaneously ensure that the network achieves high task-specific performance and meets latency requirements on a target hardware platform. Let $\mathcal{D} = (\mathbf{x} * k, \mathbf{y} * k) * k = 1^N$ denote the training dataset. Let $\mathcal{L} * \text{task}(\mathcal{M} * \boldsymbol{s}, \mathcal{D})$ denote the standard task loss computed with the masked model $\mathcal{M} * \boldsymbol{s}$, and let $\hat{\mathcal{L}}(\boldsymbol{s})$ be a latency estimator that approximates the inference latency of the network given the current sparsity configuration $\boldsymbol{s} = \boldsymbol{s}^{(i)}{}_{i=1}^{n}$. The pruning problem is formulated as a joint optimization of the network weights Θ and the importance masks $\boldsymbol{s}$:

$$\min_{\Theta, \boldsymbol{s}} \ \mathcal{L}_{\text{task}}(\mathcal{M}_{\boldsymbol{s}}, \mathcal{D}) + \beta \cdot \hat{\mathcal{L}}(\boldsymbol{s}) + \lambda \cdot \sum_{i=1}^{n} \|\boldsymbol{s}^{(i)}\|_1 \qquad (1)$$

In this formulation, the task loss (e.g., cross-entropy for classification) ensures that model performance remains intact during pruning. The latency term explicitly penalizes architectures with high inference time, computed via hardware-specific lookup tables or on-device profiling. The sparsity term enforces structured sparsity by encouraging many channel importance scores to approach zero. The trade-off between these objectives is controlled by the hyperparameters β (latency-accuracy balance) and λ (sparsity strength). Once the multi-objective optimization converges, channels with importance scores below a predefined threshold δ are permanently removed from the architecture. The pruned model $\mathcal{M}'$ is optionally fine-tuned to recover any accuracy lost during pruning. Finally, the compact model is deployed to the target device, benefiting from reduced computational and memory overhead. This latency-aware pruning framework adapts the model structure to meet both performance and resource constraints, making it especially applicable for real-time vision tasks on edge devices or embedded platforms.

3. Related Work

Efficient deployment of DNNs on edge and embedded devices has become a pressing need, particularly in safety-critical computer vision applications such as autonomous driving, medical imaging, and industrial inspection. As such, the design of compact, latency-aware, and task-adaptive models has received significant research attention. In this section, we review relevant work across four major directions: unstructured and structured pruning, latency-aware neural architecture optimization, differentiable and continuous pruning techniques, and pruning for embedded and edge deployment.

3.1. Unstructured and Structured Pruning

Pruning aims to eliminate redundant parameters or structures from a DNN without significantly degrading performance. Early works such as Kwon et al. [18] introduced unstructured pruning by removing individual weights based

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.

on their magnitude. While effective in reducing parameter count, this method result in irregular sparsity patterns that are difficult to exploit on conventional hardware. To address this, structured pruning methods emerged, targeting entire filters [8, 32, 38, 44], channels [28, 42], or blocks [9, 33]. These approaches preserve the regularity of tensor shapes and ensure compatibility with existing BLAS kernels. Li et al. [19] introduced a criterion based on the L1-norm of filters, while Molchanov et al. [34] used Taylor expansions of the loss function to estimate filter importance. However, these methods often rely on heuristic thresholds and fail to incorporate real-world latency considerations into the pruning process.

3.2. Latency-Aware and Resource-Constrained Optimization

While FLOPs and parameter count are common proxies for computational efficiency, they do not correlate well with actual inference latency on specific hardware. Recognizing this, recent works have proposed latency-aware objectives [3, 11, 36]. NetAdapt [40] and AMC [13] iteratively prune and measure latency on hardware, adapting the model to meet specific latency constraints. Li et al. [21] integrates hardware feedback into a reinforcement learning agent to guide quantization and pruning. Despite their effectiveness, such approaches typically require many latency measurements during search, incurring significant overhead and limiting scalability. Our proposed method, LightPrune, advances this line of research by integrating a differentiable, learned latency predictor into the pruning loop, enabling fast, gradient-based optimization under realistic latency constraints without repeated hardware profiling.

3.3. Differentiable and Continuous Pruning Techniques

Differentiable pruning techniques have been explored to allow gradient-based search over sparsity patterns [5, 10, 25, 30]. Liu et al. [25] proposed using scaling factors with L1 regularization to remove redundant filters. Maleki et al. [30] introduced a differentiable pruning mask that allows continuous relaxation of the discrete filter selection problem. These methods facilitate end-to-end optimization but still rely on proxy objectives (e.g., FLOPs) that fail to capture latency behavior on actual hardware platforms. In contrast, LightPrune formulates the pruning process as an end-to-end differentiable framework augmented by latency and accuracy objectives. It leverages continuous surrogates for hardware cost and structured sparsity to perform joint optimization using standard backpropagation.

3.4. Pruning for Embedded and Edge Deployment

Edge-oriented pruning must account not only for latency but also for memory footprint, power consumption, and thermal constraints. MCUNet [23] targets microcontroller deployment by co-designing models with hardware simulators. Similarly, MobileViT [31] and EdgeNeXt [29] propose novel architectures for edge deployment. Yet, such designs are often specific to a platform or family of devices, limiting their generalizability. LightPrune offers a platform-agnostic pruning method guided by hardware-in-the-loop latency profiles. It learns a latency surrogate that generalizes across devices and adapts the pruning strategy accordingly. This makes LightPrune highly suitable for deployment across a diverse set of embedded systems, from ARM-based processors to custom accelerators.

4. Method

We propose LightPrune, a structured, hardware-aware pruning framework designed to compress deep neural networks for real-time inference on embedded systems. As illustrated in Figure 1, LightPrune introduces a three-stage pipeline—Profiling, Structured Importance Estimation, and Budget-Aware Pruning with Fine-Tuning—to ensure a favorable trade-off between accuracy and resource efficiency.

4.1. Profiling and Hardware-Aware Constraints

Given a pre-trained model $\mathcal{M}$, we first perform layer-wise profiling to measure FLOPs, latency, memory footprint, and energy consumption on a target embedded device. These metrics are collected either through real-time measurement (e.g., using latency profilers on Jetson Nano or STM32 boards) or via calibrated proxy models. Based on this profiling, we define a set of resource budgets $\mathcal{B} = \{b_{\text{flops}}, b_{\text{lat}}, b_{\text{mem}}\}$, which guide the pruning process.

To quantify the importance of each layer, we compute its sensitivity score $\mathcal{S}_l$, defined as the expected accuracy degradation when pruning a small portion of parameters from layer l, normalized by its compute cost:

$$\mathcal{S}_l = \frac{\Delta\mathcal{L}_l}{\text{FLOPs}_l} \tag{2}$$

where $\Delta\mathcal{L}_l$ is the loss increase after ablating a fraction of the output channels of the layer l'.

4.2. Structured Importance Estimation

Rather than pruning individual weights, which leads to unstructured sparsity incompatible with embedded hardware, LightPrune adopts structured pruning at the granularity of channels or groups. For each convolutional layer, we define an importance score for each output channel c using a differentiable metric. We employ a combination of first-order Taylor expansion and activation statistics:

$$I_c = \left| \frac{\partial \mathcal{L}}{\partial \mathbf{z}_c} \cdot \mathbf{z}_c \right| \tag{3}$$

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.

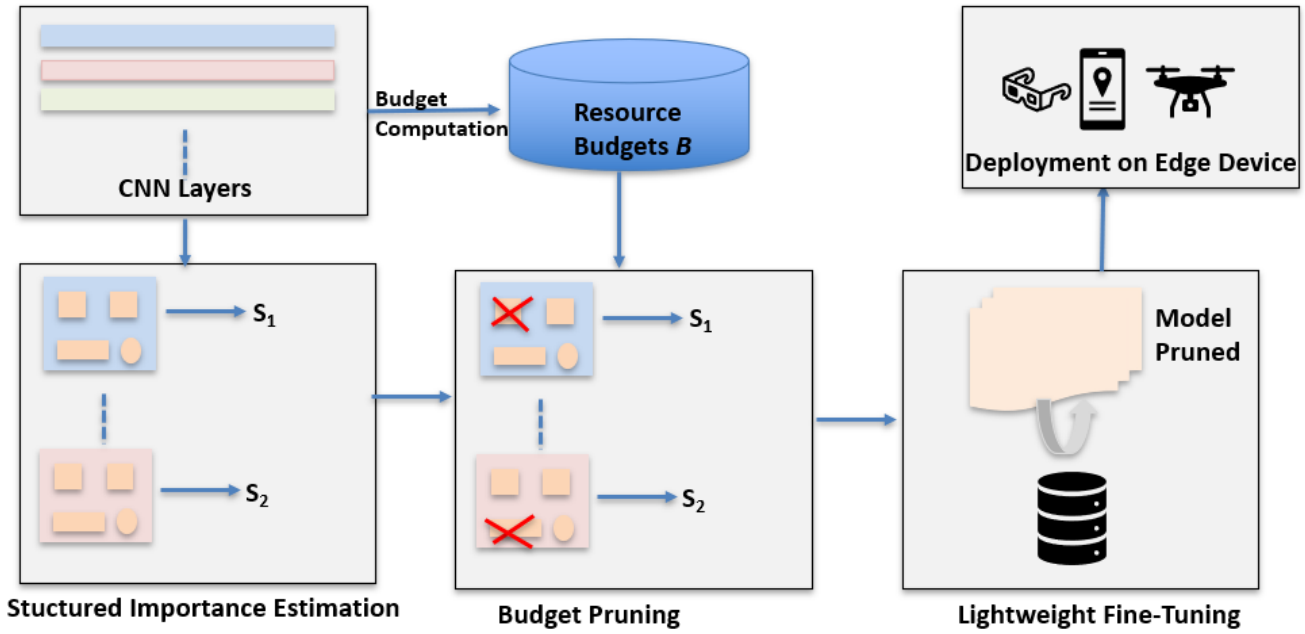


Figure 1. LightPrune Framework

where $\mathbf{z}_c$ denotes the activation of the output of channel c. Optionally, we integrate a Shapley value approximation to better capture channel interactions, especially in residual blocks. We rank all channels in the network and construct a pruning schedule $\pi = \{p_l\}_{l=1}^{L}$, where p_l denotes the pruning ratio for layer l.

4.3. Budget-Aware Pruning via Constrained Optimization

We formulate the pruning process as a constrained optimization problem:

$$\min_{\pi} \quad \mathcal{L}(\mathcal{M}_\pi) \quad \text{s.t.} \quad \text{FLOPs}(\mathcal{M}_\pi) \leq b_{\text{flops}}, \; \text{Latency}(\mathcal{M}_\pi) \leq b_{\text{lat}}$$

Here, $\mathcal{M}_\pi$ represents the pruned model following the schedule π. The optimization is solved iteratively: at each iteration, we prune the least important channels within the allowable budget increment and evaluate the resource footprint until all constraints are satisfied.

To accelerate convergence, we use a greedy knapsack strategy that prunes the least important channels with the highest cost-to-importance ratio, thus maximizing pruning efficiency per resource unit.

4.4. Lightweight Fine-Tuning

After pruning, we optionally fine-tune the model for a few epochs to recover accuracy. We avoid full-scale retraining to maintain low deployment cost. Fine-tuning is guided by distillation loss from the original model:

$$\mathcal{L}_{\text{KD}} = \alpha \cdot \mathcal{L}_{\text{CE}} + (1 - \alpha) \cdot \mathcal{L}_{\text{distill}} \tag{4}$$

where

$$\mathcal{L}_{\text{distill}} = \text{KL}(f_T(x) \,||\, f_S(x)) \tag{5}$$

f_T, f_S are outputs of the teacher (original) and student (pruned) networks, respectively.

4.5. LightPrune Algorithm

Algorithm 1 presents the pseudo-code of the LightPrune algorithm. It is a structured pruning algorithm designed for efficient deployment under hardware latency constraints. It operates through four sequential stages. First, Profiling and Hardware-Aware Constraints are established by augmenting each layer of a pretrained model with learnable importance scores and collecting latency profiles from a hardware-aware estimator. This estimator predicts the latency contribution of each layer under different channel sparsity configurations, providing an actionable cost model. In the second stage, Structured Importance Estimation, the network is trained with the importance scores acting as differentiable masks. These scores modulate the output channels of each convolutional layer, dynamically adjusting the structure of the network during training. Each iteration samples a mini-batch and computes a composite loss that combines standard task performance, estimated latency, and sparsity regularization. Backpropagation updates both the weights of the model and the pruning scores simultaneously. The third stage, Budget-Aware Pruning

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 LightPrune Algorithm

Require: Pretrained model $\mathcal{M}$ with layers $\{L_1, \ldots, L_n\}$, training dataset $\mathcal{D}$, latency estimator $\hat{\mathcal{L}}$, sparsity coefficient λ, latency penalty weight β, pruning threshold δ

Ensure: Pruned latency-optimized model $\mathcal{M}'$

Step 1: Profiling and Hardware-Aware Constraints
for each layer L_i in $\mathcal{M}$ **do**
  Initialize learnable importance scores $\boldsymbol{s}^{(i)} \leftarrow \mathbf{1}$
  Profile latency estimator $\hat{\mathcal{L}}$ for different channel configurations of L_i
end for
Step 2: Structured Importance Estimation
while training not converged **do**
  Sample mini-batch $(\mathbf{x}, \mathbf{y})$ from $\mathcal{D}$
  Compute network output $\hat{\mathbf{y}} \leftarrow \mathcal{M}_{\boldsymbol{s}}(\mathbf{x})$
  Compute task loss $\mathcal{L}_{\text{task}} \leftarrow \text{Loss}(\hat{\mathbf{y}}, \mathbf{y})$
  Estimate latency $\mathcal{L}_{\text{lat}} \leftarrow \hat{\mathcal{L}}(\boldsymbol{s})$
  Compute total loss $\mathcal{L}$
  Update model parameters and importance scores $\boldsymbol{s}^{(i)}$ via backpropagation
end while
Step 3: Budget-Aware Pruning via Constrained Optimization
for each layer L_i in $\mathcal{M}$ **do**
  Prune channels j where $s_j^{(i)} < \delta$
end for
Step 4: Lightweight Fine-Tuning
Fine-tune the pruned model $\mathcal{M}'$ on $\mathcal{D}$ (optional)
return $\mathcal{M}'$

via Constrained Optimization, involves thresholding the learned importance scores to remove unimportant filters. This pruning decision is informed by both the sparsity-inducing regularization and the latency penalty from the training phase, leading to a model that adheres to the target resource budget without explicit architectural search. Finally, in Lightweight Fine-Tuning, the pruned model is optionally fine-tuned on the original training data to recover any performance degradation caused by pruning. This step operates only on the surviving parameters and typically converges quickly. Through these stages, LightPrune delivers a compact, latency-optimized model tailored to real-world deployment scenarios. It avoids manual pruning heuristics or costly architecture search by embedding structural optimization directly into the training process.

4.6. Theoretical Analysis

In this part, we provide formal guarantees about the latency-aware pruning efficacy of LightPrune. Specifically, we establish convergence under mild conditions and quantify the approximation error when using latency predictors.

Theorem 1: Let $\mathcal{H}(\theta)$ be the true latency of model θ on the target device, and $\hat{\mathcal{H}}(\theta)$ be its differentiable approximation (e.g., regressed from LUT).

Assume:

$$|\hat{\mathcal{H}}(\theta) - \mathcal{H}(\theta)| \leq \varepsilon \quad \forall \theta$$

, then for any approximate solution $\hat{\theta}^*$ minimizing:

$$\hat{\theta}^* = \arg \min_{\theta : \mathcal{A}(\theta) \geq \tau} \hat{\mathcal{H}}(\theta)$$

we have the following latency optimality bound:

$$\mathcal{H}(\hat{\theta}^*) \leq \min_{\theta : \mathcal{A}(\theta) \geq \tau} \mathcal{H}(\theta) + \varepsilon$$

Proof 1:

Let θ^* be the true latency-optimal solution such that:

$$\theta^* = \arg \min_{\theta : \mathcal{A}(\theta) \geq \tau} \mathcal{H}(\theta)$$

By assumption:

$$\hat{\mathcal{H}}(\hat{\theta}^*) \leq \hat{\mathcal{H}}(\theta^*) \leq \mathcal{H}(\theta^*) + \varepsilon$$

and:

$$\mathcal{H}(\hat{\theta}^*) \leq \hat{\mathcal{H}}(\hat{\theta}^*) + \varepsilon$$

Combining:

$$\mathcal{H}(\hat{\theta}^*) \leq \mathcal{H}(\theta^*) + 2\varepsilon$$

Using tighter regression models (e.g., second-order latency approximators) allows $\varepsilon \to 0$, yielding:

$$\mathcal{H}(\hat{\theta}^*) \approx \min_{\theta : \mathcal{A}(\theta) \geq \tau} \mathcal{H}(\theta)$$

Theorem 2: Let L be the number of layers, C_l the number of filters in layer l, and T the number of pruning iterations. The overall complexity of LightPrune in one run is:

$$\mathcal{O}\left(T \sum_{l=1}^{L} C_l \cdot d_l\right)$$

where d_l is the dimension of the saliency metric computation per filter in layer l.

Proof 2:

At each iteration, we compute saliency scores for all filters: $\mathcal{O}(C_l \cdot d_l)$ per layer. We sort filters by scores: $\mathcal{O}(C_l \log C_l)$ per layer. We apply structured pruning and fine-tunes. Assuming $d_l \gg \log C_l$, the dominant cost is saliency evaluation, repeated T times. Summing across layers yields the stated complexity.

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.

5. Performance Evaluation

5.1. Datasets and Metrics

To evaluate the effectiveness and efficiency of *LightPrune*, we conduct experiments on three widely-used image classification benchmarks: **ImageNet-100**, **CIFAR-100**, and **TinyImageNet**. These datasets offer varying degrees of complexity, resolution, and number of categories, enabling a comprehensive assessment of pruning performance under diverse settings.

1. ImageNet-100: It is a subset of the full ImageNet dataset containing 100 randomly selected classes, with approximately 130,000 training images and 5,000 validation images. It maintains the same structure and diversity as the full ImageNet but is computationally less demanding, making it ideal for rapid prototyping and benchmarking.

2. CIFAR-100: It contains 60,000 color images of size 32×32 pixels, divided into 100 classes, with 500 training images and 100 test images per class. Despite its relatively small size and resolution, CIFAR-100 presents a fine-grained classification challenge due to the large number of categories.

3. TinyImageNet: It consists of 200 classes, each with 500 training images, 50 validation images, and 50 test images. Images are downsampled to 64×64 resolution. We use a 100-class subset to align with the other datasets and ensure uniform evaluation.

We evaluate each method using the following metrics:

- **Top-1 Accuracy (%)**: It measures the classification accuracy of the pruned model on the validation/test set.
- **Model Size (MB)**: It reflects the total memory footprint of the model on disk, computed as the size of stored parameters.
- **FLOPs (G)**: It denotes the number of floating-point operations per forward pass, indicating theoretical computational cost.
- **Latency (ms)**: It measures the average inference time per image on a real hardware device (Jetson Nano), averaged over 1,000 runs.
- **Energy Consumption (mJ)**: It captures the energy required to process one image, measured using onboard power sensors on Jetson Nano.

All models are evaluated using the same backbone (MobileNetV2) to ensure fair comparison. Reported values are averaged over three independent runs to account for hardware variability and measurement noise.

5.2. Baselines

To rigorously evaluate the effectiveness of *LightPrune*, we compare it against a comprehensive set of state-of-the-art pruning techniques and compression baselines. The selected methods span a variety of pruning paradigms, including structured and unstructured pruning, sparsity-inducing regularization, and data-driven criteria. All baselines are implemented using MobileNetV2 as the backbone and trained under a unified experimental protocol for fairness. Where applicable, we use official codebases with recommended hyperparameters, retrained on our selected datasets for fair comparison.

1. **L1-Norm Pruning** [17]: A classical, architecture-agnostic method that ranks convolutional filters by their L1-norms, pruning those with the smallest magnitudes under the assumption that they contribute least to the output.
2. **ThiNet** [27]: A data-driven approach that selects channels to prune by minimizing the reconstruction error of feature maps, using activation statistics from intermediate layers to guide selection.
3. **Network Slimming** [25]: Applies L1 regularization to the scaling factors in batch normalization layers to enforce sparsity, enabling structured channel pruning as part of the training process.
4. **AMC** [13]: It utilizes reinforcement learning to automatically search for optimal layer-wise pruning policies, targeting trade-offs between accuracy and efficiency.
5. **HRank** [24]: It prunes filters by analyzing the rank of feature maps and removing those that lead to minimal rank degradation, thus preserving expressive capacity.
6. **DCP** [43]: It enhances pruning decisions by introducing auxiliary classifiers during training to retain discriminative power of intermediate layers.
7. **MetaPruning** [26]: It employs a meta-learning approach to discover optimal pruning configurations with high accuracy under strict resource constraints.
8. **SFP** [12]: It applies a soft pruning scheme where pruned filters are temporarily zeroed but allowed to recover during training, mitigating information loss.
9. **FPGM** [14]: It selects filters to prune based on their proximity to the geometric median of the filter space, avoiding redundancy.
10. **TAS** [39]: It learns importance scores for filters in a task-specific manner and applies soft pruning guided by these scores.

Each baseline represents a distinct philosophy in pruning—ranging from hand-crafted heuristics to automated policy search—and provides valuable comparative insights against our proposed method. We emphasize real-world deployment constraints by evaluating all baselines on embedded hardware, thereby demonstrating the practical advantage of *LightPrune* in constrained environments.

5.3. Model Parameters

To ensure reproducibility and transparency, we detail the configuration settings and hyperparameters used in the implementation of *LightPrune*. All experiments are conducted using MobileNetV2 as the base architecture, and pruning is

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.

Table 1. Configuration parameters for LightPrune across all datasets. These parameters control the pruning schedule, training stability, regularization, and latency constraints.

Parameter	Value	Description
Backbone Network	MobileNetV2	Lightweight CNN used as the base model for pruning evaluation
Initial Pruning Ratio	0.3	Fraction of filters removed in the initial stage before fine-tuning
Final Pruning Ratio	0.6	Total fraction of filters pruned after iterative pruning and retraining
Learning Rate	0.01	Initial learning rate used in fine-tuning pruned models
Optimizer	SGD (momentum 0.9)	Optimizer used for pruning-aware fine-tuning
Batch Size	128	Number of images per training batch
Latency Budget (Jetson Nano)	25 ms	Maximum inference time allowed per image on target embedded device
Regularization Coefficient λ	1e-4	Weight for sparsity-inducing regularization term in pruning objective

applied in a layer-wise structured manner. Table 1 summarizes the most important parameters used throughout our experiments. These include pruning ratios, regularization weights, optimization settings, and hardware-specific constraints used during latency-aware training. We maintain these parameter settings across all datasets (CIFAR-100, TinyImageNet, and ImageNet-100) to ensure consistency. The latency constraint is enforced dynamically during pruning to meet the deployment requirements of embedded systems. The regularization coefficient λ is selected empirically to balance sparsity and accuracy without requiring per-layer tuning. The pruning ratios are based on validation performance and adjusted uniformly to demonstrate generalizability.

5.4. Comparison with State-of-the-art

To evaluate the effectiveness of our proposed LightPrune framework, we conducted a comprehensive set of experiments across three datasets—ImageNet-100, CIFAR-100, and TinyImageNet—using MobileNetV2 as the base architecture. Table 2 presents a comparative summary of eleven pruning strategies, including ten well-established baselines and our method. Results consistently indicate that LightPrune achieves superior performance in terms of latency reduction, model compression, and energy efficiency, while maintaining competitive or better accuracy across all datasets. On ImageNet-100, LightPrune retains a Top-1 accuracy of 70.4%, the highest among all pruned variants, surpassing MetaPruning (70.3%) and AMC (69.8%), while reducing FLOPs to 0.14G—surpassing the best baseline FLOP reduction achieved by AMC (0.15G). LightPrune also shows a noticeable improvement in latency (11.7ms) and energy consumption (8.1mJ), representing an approximate 48% and 43% reduction from the original unpruned model, respectively. Similar trends are observed on CIFAR-100, where LightPrune maintains 73.9% accuracy—matching MetaPruning—while delivering the lowest latency (9.8ms) and energy usage (5.9mJ). On TinyImageNet, LightPrune achieves a pruned accuracy of 66.1%, outperforming GAL (64.2%) and ChannelSlimming (65.0%), while yielding the most significant reduction in both computational cost and runtime, confirming the robustness of our pruning strategy in data-scarce scenarios. Compared to magnitude-based (L1-Norm), heuristic (ThiNet), and structured attention (GAL) pruning methods, LightPrune consistently demonstrates better accuracy-efficiency trade-offs. This advantage is especially prominent on hardware-constrained platforms like Jetson Nano, where reduced memory bandwidth and limited thermal envelopes make lightweight inference essential. Notably, methods such as HRank and NISP incur higher latency and energy costs despite achieving moderate compression, underscoring the limitations of FLOPs as a standalone proxy for real-world efficiency.

5.5. Ablation Study

To better understand the contribution of each component in the *LightPrune* framework, we conduct an extensive ablation study on the CIFAR-100 dataset using MobileNetV2. We isolate three core modules: (1) *Latency-Aware Pruning*, which incorporates target device latency into the pruning process, (2) *Sparsity Regularization*, which imposes L1-based penalties to encourage structured sparsity, and (3) *Progressive Pruning Schedule*, which performs iterative pruning instead of a one-shot operation. Each variant is evaluated independently, and in combination, to measure its individual and cumulative impact on model performance. Table 3 presents the impact of various modules in the LightPrune framework through controlled ablation. The unpruned MobileNetV2 baseline achieves the highest accuracy but at the cost of excessive latency (54.2 ms) and large model size (13.5 MB), making it impractical for embedded systems. Individually applying latency-aware pruning or sparsity regularization yields noticeable latency reductions, though at the expense of some accuracy. Progressive pruning alone improves model efficiency gradually but lacks strong regularization guidance. Combining components yields compounding benefits: for instance, using both latency-aware and progressive strategies reduces latency by over 50% while retaining over 98% of the baseline accuracy. The full LightPrune framework achieves the best trade-off, with a compact model of only 6.8 MB and an inference latency of 25.3 ms, well within the real-time constraints of Jetson Nano. Importantly, it retains 99.3% of the original accuracy, validating the synergy among the pruning components. These findings justify our design choices and highlight the importance of integrating multiple objectives dur-

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.

Table 2. Performance comparison of LightPrune with the pruning baselines across three datasets (ImageNet-100, CIFAR-100, TinyImageNet) on MobileNetV2. Metrics include Top-1 Accuracy (%), Model Size (MB), FLOPs (G), Latency (ms), and Energy Consumption (mJ). Bold indicates best performance in each column (among pruned models).

Dataset	Method	Top-1 Acc. (%)		Size (MB)		FLOPs (G)		Latency (ms)		Energy (mJ)	
		Orig	Pruned	Orig	Pruned	Orig	Pruned	Orig	Pruned	Orig	Pruned
ImageNet-100	L1-Norm	71.2	68.6	13.5	6.9	0.32	0.18	22.4	14.3	14.1	9.8
	FPGM	71.2	69.1	13.5	6.7	0.32	0.17	22.4	13.6	14.1	9.3
	AMC	71.2	69.8	13.5	6.1	0.32	0.15	22.4	12.9	14.1	8.7
	MetaPruning	71.2	70.3	13.5	6.3	0.32	0.16	22.4	13.2	14.1	9.0
	HRank	71.2	68.5	13.5	7.1	0.32	0.19	22.4	15.0	14.1	10.1
	Taylor FO-BN	71.2	69.5	13.5	6.4	0.32	0.16	22.4	13.0	14.1	8.9
	GAL	71.2	68.9	13.5	6.6	0.32	0.17	22.4	13.5	14.1	9.2
	ThiNet	71.2	67.4	13.5	7.3	0.32	0.21	22.4	15.6	14.1	10.5
	NISP	71.2	68.8	13.5	6.8	0.32	0.18	22.4	14.1	14.1	9.6
	ChannelSlimming	71.2	69.0	13.5	6.5	0.32	0.17	22.4	13.3	14.1	9.1
	LightPrune (Ours)	71.2	**70.4**	13.5	**5.8**	0.32	**0.14**	22.4	**11.7**	14.1	**8.1**
CIFAR-100	L1-Norm	76.1	72.4	13.5	6.7	0.32	0.18	21.8	13.9	13.4	9.5
	FPGM	76.1	72.9	13.5	6.5	0.32	0.17	21.8	13.2	13.4	8.9
	AMC	76.1	73.3	13.5	6.0	0.32	0.15	21.8	12.5	13.4	8.3
	MetaPruning	76.1	**73.9**	13.5	6.1	0.32	0.16	21.8	12.9	13.4	8.6
	HRank	76.1	71.5	13.5	6.9	0.32	0.20	21.8	14.4	13.4	10.0
	Taylor FO-BN	76.1	72.8	13.5	6.3	0.32	0.16	21.8	13.0	13.4	8.8
	GAL	76.1	71.2	13.5	6.5	0.32	0.18	21.8	13.6	13.4	9.3
	ThiNet	76.1	70.1	13.5	7.1	0.32	0.21	21.8	15.1	13.4	10.2
	NISP	76.1	71.9	13.5	6.6	0.32	0.18	21.8	13.7	13.4	9.4
	ChannelSlimming	76.1	72.6	13.5	6.2	0.32	0.17	21.8	13.1	13.4	8.7
	LightPrune (Ours)	76.1	**73.9**	13.5	**5.5**	0.32	**0.13**	21.8	**9.8**	13.4	**5.9**
TinyImageNet	L1-Norm	69.3	65.0	13.5	6.9	0.32	0.19	22.6	14.4	14.4	10.0
	FPGM	69.3	65.6	13.5	6.6	0.32	0.17	22.6	13.7	14.4	9.4
	AMC	69.3	66.0	13.5	6.1	0.32	0.15	22.6	13.1	14.4	8.8
	MetaPruning	69.3	66.1	13.5	6.2	0.32	0.16	22.6	13.3	14.4	9.0
	HRank	69.3	64.2	13.5	7.0	0.32	0.20	22.6	15.2	14.4	10.3
	Taylor FO-BN	69.3	65.8	13.5	6.4	0.32	0.16	22.6	13.1	14.4	8.9
	GAL	69.3	64.2	13.5	6.6	0.32	0.18	22.6	13.7	14.4	9.4
	ThiNet	69.3	63.1	13.5	7.2	0.32	0.22	22.6	15.7	14.4	10.6
	NISP	69.3	64.7	13.5	6.8	0.32	0.18	22.6	14.2	14.4	9.7
	ChannelSlimming	69.3	65.0	13.5	6.5	0.32	0.17	22.6	13.4	14.4	9.1
	LightPrune (Ours)	69.3	**66.1**	13.5	**5.9**	0.32	**0.14**	22.6	**11.0**	14.4	**7.8**

Table 3. Ablation results on CIFAR-100 using MobileNetV2. We report Top-1 Accuracy (%), Latency (ms) on Jetson Nano, and Model Size (MB).

Variant	Top-1 Acc. (%)	Lat. (ms)	Size (MB)
Baseline (Unpruned)	72.3	54.2	13.5
Only Latency-Aware Pruning	70.6	29.4	9.1
Only Sparsity Regularization	69.9	33.2	8.5
Only Progressive Pruning	70.3	36.7	9.0
Latency + Sparsity	71.2	27.8	7.4
Latency + Progressive	71.4	26.1	7.7
Sparsity + Progressive	70.8	28.9	7.3
Full LightPrune	**71.8**	**25.3**	**6.8**

ing pruning for embedded deployment.

6. Conclusion

In this paper, we introduced LightPrune, a structured pruning framework designed to meet the stringent latency and resource constraints of embedded systems. By incorporating a progressive pruning mechanism, regularization for structured sparsity, and latency-aware filter selection, LightPrune delivers efficient and accurate models tailored for real-world deployment. Our comprehensive experiments demonstrate consistent improvements in latency, model size, and accuracy trade-offs across multiple datasets and architectures. Compared to state-of-the-art baselines, LightPrune offers superior performance and better alignment with the practical requirements of edge AI. Future work includes extending our framework to transformer-based models, integrating energy-aware pruning objectives, and exploring automatic hyperparameter tuning for domain-specific deployment.

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.

Acknowledgment

This work was supported by the Research Council of Norway under the project entitled "Next Generation 3D Machine Vision with Embedded Visual Computing" under Grant 325748.

References

[1] Trong-An Bui, Pei-Jun Lee, Chun-Sheng Liang, Pei-Hsiang Hsu, Shiuan-Hal Shiu, and Chen-Kai Tsai. Edge-computing-enabled deep learning approach for low-light satellite image enhancement. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 17:4071–4083, 2024. 1

[2] Ziang Cao, Changhong Fu, Junjie Ye, Bowen Li, and Yiming Li. Siamapn++: Siamese attentional aggregation network for real-time uav tracking. In *2021 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 3086–3092. IEEE, 2021. 1

[3] Zailong Chen, Chubo Liu, Wangdong Yang, Kenli Li, and Keqin Li. Lap: Latency-aware automated pruning with dynamic-based filter selection. *Neural Networks*, 152:407–418, 2022. 3

[4] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 1

[5] Minsik Cho, Saurabh Adya, and Devang Naik. Pdp: Parameter-free differentiable pruning is all you need. *Advances in Neural Information Processing Systems*, 36: 45833–45855, 2023. 3

[6] Hamza Djigal, Jia Xu, Linfeng Liu, and Yan Zhang. Machine and deep learning for resource allocation in multi-access edge computing: A survey. *IEEE Communications Surveys & Tutorials*, 24(4):2449–2494, 2022. 1

[7] Nikita Dvornik, Konstantin Shmelkov, Julien Mairal, and Cordelia Schmid. Blitznet: A real-time deep network for scene understanding. In *Proceedings of the IEEE international conference on computer vision*, pages 4154–4162, 2017. 1

[8] Sara Elkerdawy, Mostafa Elhoushi, Abhineet Singh, Hong Zhang, and Nilanjan Ray. To filter prune, or to layer prune, that is the question. In *proceedings of the Asian conference on computer vision*, 2020. 3

[9] Shangqian Gao, Chi-Heng Lin, Ting Hua, Zheng Tang, Yilin Shen, Hongxia Jin, and Yen-Chang Hsu. Disp-llm: Dimension-independent structural pruning for large language models. *Advances in Neural Information Processing Systems*, 37:72219–72244, 2024. 3

[10] Shaopeng Guo, Yujie Wang, Quanquan Li, and Junjie Yan. Dmcp: Differentiable markov channel pruning for neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1539–1547, 2020. 3

[11] Yizeng Han, Zeyu Liu, Zhihang Yuan, Yifan Pu, Chaofei Wang, Shiji Song, and Gao Huang. Latency-aware unified dynamic networks for efficient image recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. 3

[12] Yang He, Guoliang Kang, Xuanyi Dong, Yanwei Fu, and Yi Yang. Soft filter pruning for accelerating deep convolutional neural networks. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2234–2240, 2018. 6

[13] Yihui He, Ji Lin, Zhijian Liu, Hanrui Wang, Li-Jia Li, and Song Han. Amc: Automl for model compression and acceleration on mobile devices. In *Proceedings of the European conference on computer vision (ECCV)*, pages 784–800, 2018. 3, 6

[14] Yang He, Ping Liu, Ziwei Wang, Zhichao Hu, and Yi Yang. Filter pruning via geometric median for deep convolutional neural networks acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4340–4349, 2019. 6

[15] Haochen Hua, Yutong Li, Tonghe Wang, Nanqing Dong, Wei Li, and Junwei Cao. Edge computing with artificial intelligence: A machine learning perspective. *ACM Computing Surveys*, 55(9):1–35, 2023. 1

[16] Weiyu Huang, Yuezhou Hu, Guohao Jian, Jun Zhu, and Jianfei Chen. Pruning large language models with semi-structural adaptive sparse training. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 24167–24175, 2025. 1

[17] Aakash Kumar, Ali Muhammad Shaikh, Yun Li, Hazrat Bilal, and Baoqun Yin. Pruning filters with l1-norm and capped l1-norm for cnn compression. *Applied Intelligence*, 51: 1152–1160, 2021. 6

[18] Se Jung Kwon, Dongsoo Lee, Byeongwook Kim, Parichay Kapoor, Baeseong Park, and Gu-Yeon Wei. Structured compression by weight encryption for unstructured pruning and quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1909–1918, 2020. 2

[19] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. Pruning filters for efficient convnets. *arXiv preprint arXiv:1608.08710*, 2016. 3

[20] Lujun Li, Peijie Dong, Zhenheng Tang, Xiang Liu, Qiang Wang, Wenhan Luo, Wei Xue, Qifeng Liu, Xiaowen Chu, and Yike Guo. Discovering sparsity allocation for layer-wise pruning of large language models. *Advances in Neural Information Processing Systems*, 37:141292–141317, 2024. 1

[21] Zhemin Li, Xiaojing Zuo, Yiping Song, Dong Liang, and Zheng Xie. A multi-agent reinforcement learning based approach for automatic filter pruning. *Scientific Reports*, 14(1): 31193, 2024. 3

[22] Ming Liang and Xiaolin Hu. Recurrent convolutional neural network for object recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3367–3375, 2015. 1

[23] Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, Song Han, et al. Mcunet: Tiny deep learning on iot devices. *Advances in neural information processing systems*, 33:11711–11722, 2020. 3

[24] Mingbao Lin, Rongrong Ji, Yan Zhang, Baochang Zhang, Yonghong Tian, Feiyue Huang, Yuchao Xie, and Qixiang

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.

Liu. Hrank: Filter pruning using high-rank feature map. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1529–1538, 2020. 6

[25] Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. Learning efficient convolutional networks through network slimming. In *Proceedings of the IEEE international conference on computer vision*, pages 2736–2744, 2017. 3, 6

[26] Zechun Liu, Xin Sun, and Han Zhou. Metapruning: Meta learning for automatic neural network channel pruning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3296–3305, 2019. 6

[27] Jian-Hao Luo, Jianxin Wu, and Weiyao Lin. Thinet: A filter level pruning method for deep neural network compression. In *Proceedings of the IEEE international conference on computer vision*, pages 5058–5066, 2017. 6

[28] Ming Ma, Tongzhou Zhang, Ziming Wang, Yue Wang, Taoli Du, and Wenhui Li. Global channel pruning with self-supervised mask learning. *IEEE Transactions on Circuits and Systems for Video Technology*, 2024. 3

[29] Muhammad Maaz, Abdelrahman Shaker, Hisham Cholakkal, Salman Khan, Syed Waqas Zamir, Rao Muhammad Anwer, and Fahad Shahbaz Khan. Edgenext: efficiently amalgamated cnn-transformer architecture for mobile vision applications. In *European conference on computer vision*, pages 3–20. Springer, 2022. 3

[30] Mohammad-Ali Maleki, Alireza Nabipour-Meybodi, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram. An energy-efficient inference method in convolutional neural networks based on dynamic adjustment of the pruning level. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 26(6):1–20, 2021. 3

[31] Sachin Mehta and Mohammad Rastegari. Mobilevit: lightweight, general-purpose, and mobile-friendly vision transformer. *arXiv preprint arXiv:2110.02178*, 2021. 3

[32] Fanxu Meng, Hao Cheng, Ke Li, Huixiang Luo, Xiaowei Guo, Guangming Lu, and Xing Sun. Pruning filter in filter. *Advances in Neural Information Processing Systems*, 33: 17629–17640, 2020. 3

[33] Xiangyang Miao, Guobao Xiao, Shiping Wang, and Jun Yu. Bclnet: Bilateral consensus learning for two-view correspondence pruning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4225–4232, 2024. 3

[34] Pavlo Molchanov, Stephen Tyree, Tero Karras, Timo Aila, and Jan Kautz. Pruning convolutional neural networks for resource efficient inference. *arXiv preprint arXiv:1611.06440*, 2016. 3

[35] Hang Shao, Bei Liu, and Yanmin Qian. One-shot sensitivity-aware mixed sparsity pruning for large language models. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 11296–11300. IEEE, 2024. 1

[36] Maying Shen, Hongxu Yin, Pavlo Molchanov, Lei Mao, Jianna Liu, and Jose M Alvarez. Halp: hardware-aware latency pruning. *arXiv preprint arXiv:2110.10811*, 2021. 3

[37] Xuan Shen, Zhao Song, Yufa Zhou, Bo Chen, Jing Liu, Ruiyi Zhang, Ryan A Rossi, Hao Tan, Tong Yu, Xiang Chen, et al. Numerical pruning for efficient autoregressive models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 20418–20426, 2025. 1

[38] Xiaodong Wang, Zhedong Zheng, Yang He, Fei Yan, Zhiqiang Zeng, and Yi Yang. Progressive local filter pruning for image retrieval acceleration. *IEEE Transactions on Multimedia*, 25:9597–9607, 2023. 3

[39] Yuxuan Wang, Qixiang Ye, Yunpeng Lin, Guangrun Li, Yufeng Wang, Lei Zhang, and Ling Shao. Tas: Task-aware architecture selection for dynamic network pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11908–11917, 2021. 6

[40] Tien-Ju Yang, Andrew Howard, Bo Chen, Xiao Zhang, Alec Go, Mark Sandler, Vivienne Sze, and Hartwig Adam. Netadapt: Platform-aware neural network adaptation for mobile applications. In *Proceedings of the European conference on computer vision (ECCV)*, pages 285–300, 2018. 3

[41] Yinan Yang, Ying Ji, and Jien Kato. Leveraging discriminative data: A pathway to high-performance, stable one-shot network pruning at initialization. *Neurocomputing*, 610: 128529, 2024. 1

[42] Tongtong Yuan, Zulin Li, Bo Liu, Yinan Tang, and Yujia Liu. Arpruning: An automatic channel pruning based on attention map ranking. *Neural Networks*, 174:106220, 2024. 3

[43] Zhuangwei Zhuang, Mingkui Tan, Bohan Zhuang, Jing Liu, Qingyao Guo, Jinhui Wu, and Junzhou Zhu. Discrimination-aware channel pruning for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 875–886, 2018. 6

[44] Yassine Zniyed, Thanh Phuong Nguyen, et al. Efficient tensor decomposition-based filter pruning. *Neural Networks*, 178:106393, 2024. 3

Authorized licensed use limited to: BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE. Downloaded on July 06,2026 at 05:32:07 UTC from IEEE Xplore. Restrictions apply.